Generate Document

AlgoExpert · Easy · Strings

The problem

- Can `document` be built from the multiset of characters in `characters`?

Intuition

- We can build the document if, for every character, we have at least as many copies available as needed.

Brute force vs optimal

- Brute force: for each doc char, count it in both strings → O(m·(n+m))
- Optimal: one frequency map → O(n + m)
- Fail fast as soon as a count goes negative

Approach

- 1. Count available characters
- 2. For each document character, decrement its count
- 3. If any count goes negative, return false

Worked example

```
available: "Bste!hetsi ogEAxpelrt x "

document:  "AlgoExpert is the Best!"
```

Count the available characters first

Worked example




```
use 'A' → 0 left
```

Use 'A'

Worked example




```
use 'E' → 0 left
```

Use 'E'

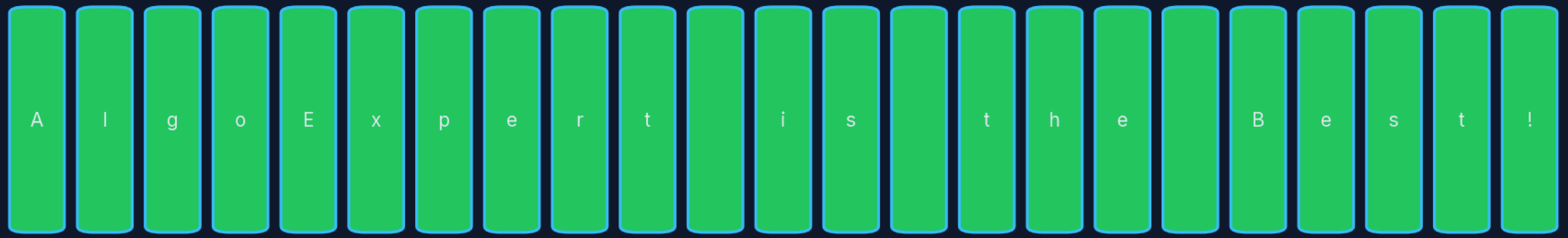
Worked example




```
use 'h' → 0 left
```

Use 'h'

Worked example



Every character was available → true

Java solution

```java
public class GenerateDocument {
    public static boolean generateDocument(String characters, String document) {
        Map<Character, Integer> counts = new HashMap<>();
        for (char c : characters.toCharArray()) counts.merge(c, 1, Integer::sum);
        for (char c : document.toCharArray()) {
            int left = counts.getOrDefault(c, 0);
            if (left == 0) return false;
            counts.put(c, left - 1);
        }
        return true;
    }
}
```

Complexity

- Time: O(n + m)
- Space: O(c)

Edge cases & pitfalls

- Case-sensitive; spaces and punctuation count
- Empty document → true
- Unicode beyond ASCII → use a map

Interview tips & follow-ups

- Ask about character set size
- Follow-up: many documents from one pool

Recap

- We can build the document if, for every character, we have at least as many copies available as needed.
- Complexity: O(n + m)